

Assignment 1

Uploading Data to the Database

Database Tuning

Report due: March 24, 2024, 23:59
PS Appointment: April 09, 2024
Grading: 5 points

Notes

- It is part of the PS to set up database systems locally on one of your machines. The reason is to avoid network latencies, which may explain strange artifacts in your measurements. That is, please ensure that client and server run on the same machine.
- This assignment involves reading the documentation of Java, the JDBC driver, and PostgreSQL. Finding the relevant sources of information is part of the challenge.
- It is suggested that you also have a look at the report template before you start working on the assignment.
- In the third part of this assignment, you are supposed to set up a second DBMS on your own machine. The recommendation is to use MariaDB. We will use the two systems in the future to investigate how different systems react to our tuning attempts.

Setting up PostgreSQL

- It is suggested to use one of the latest PostgreSQL deployments. If you want to use exactly (not required) the same versions, I use: PostgreSQL 13 with JDBC driver 4.2 in version 42.2.18.
- Nevertheless, you may use different versions of PostgreSQL. Similarly, if you prefer a different programming language for client implementation, feel free to use it.
- Installing PG Admin as development environment may also help

Support

If there are any ambiguities or problems of understanding regarding the assignment, you have the following possibilities to clarify them:

- via mail

Data

Download `dblp.zip` at:

<https://dbresearch.uni-salzburg.at/downloads/teaching/2018ss/dbt/dblp.zip>

This archive contains two tab separated files (`publ.tsv` and `auth.tsv`) that store authors and their publications as found in the DBLP¹ bibliography. The imported tables have the following schemas:

- `Auth(name(49),pubID(129))`
- `Publ(pubID(129),type(13),title(700),booktitle(132),year(4),publisher(196))`

You can assume that all attribute values are strings; the maximum string length is shown in brackets.

The straightforward algorithm to load the data from the TSV file to a table issues an SQL `INSERT` query for each line in the TSV file.

Straightforward Implementation

Implement the straightforward approach to load `auth.tsv` to the database (PostgreSQL, Java/Python).²

Efficient Approaches

The straightforward approach is slow. There are other approaches that are significantly faster. Figure out how the efficient approaches work and implement at least one of them.

Portability

Set up a different database system locally on your machine and import the data. What parts of your code do you need to change? Do you observe the same performance differences?

Report

1. Describe the two efficient approaches you implemented.
2. Give the runtime for loading `auth.tsv` with the straightforward and the efficient approaches.
3. Why are the efficient approaches faster?
4. Which tuning principle did you apply?
5. How did you ensure portability of your code?

Please indicate the average time per group member that was spent solving this assignment. The time that you indicate will have *no* impact on your grade.

¹<http://dblp.uni-trier.de/db/>

²This might be very slow. Instead of loading all the data with this approach, you can also load part of the data and assume that runtime scales linearly. Mention this in the report.

Grading scheme:

Category	Max. Points
Description of your setup	0.5
Why is the straightforward approach slow?	1.5
Why is your efficient approach faster?	1.5
Report on you portability findings.	1.5
Bonus: You found the most efficient approach	0.5

Reminder: Additional questions about the involved topics/techniques will be asked during the meeting.

Important: If the grading scheme is unclear, ask the instructor!